

CLOUD COMPUTING

COMPLETE EXAM ANSWER BANK

Part A: 5-Mark Answers
Part B: 15-Mark Answers

Prepared strictly from the supplied question bank

TABLE OF CONTENTS

PART A — 5-MARK QUESTIONS (16 questions)

PART B — 15-MARK QUESTIONS (20 questions)

FINAL REVISION CHECKLIST —

Note: Page numbers are intentionally omitted because Word pagination can change with printer/font settings.
Use Word's Navigation Pane/headings to jump between sections.

PART A

5-MARK QUESTIONS

QUESTION 1

Analyze the major issues encountered in Distributed Systems.

ANSWER (5 MARKS)

Definition:

A distributed system is a collection of independent computers that communicate over a network and cooperate so that users experience the system as a coordinated service.

Major issues:

1. Heterogeneity - different hardware, operating systems, networks and programming languages must interoperate.
2. Network failures - messages may be delayed, lost, duplicated or delivered out of order.
3. Partial failure - one component may fail while other components continue operating.
4. Concurrency - many processes execute simultaneously and may access shared resources.
5. Synchronization - distributed clocks are not perfectly synchronized; event ordering is difficult.
6. Distributed mutual exclusion - processes must coordinate access to shared resources without a central memory.
7. Deadlock - processes may wait indefinitely for resources held by one another.
8. Scalability - communication, storage and coordination overhead must remain manageable as nodes increase.
9. Security - authentication, authorization, confidentiality, integrity and availability must be protected across networks.
10. Consistency and replication - replicated data must be kept sufficiently consistent while remaining available.
11. Naming and resource discovery - services and resources must be located despite node movement or failure.
12. Load balancing - work should be distributed to avoid overloaded nodes.

Conclusion:

The main challenge is coordination without shared memory or a single global clock. Reliable communication, replication, synchronization, fault tolerance and security mechanisms are therefore essential.

QUESTION 2

Demonstrate the use of Distributed System Models in real-world applications.

ANSWER (5 MARKS)

Distributed System Models:

Distributed system models describe how components are organized, communicate and fail. Common models include client-server, peer-to-peer, multi-tier, publish-subscribe and distributed data-processing models.

1. Client-server model

Clients request services and servers provide them.

Example: A banking mobile application sends account requests to banking servers.

2. Multi-tier model

Presentation, application logic and data storage are separated into tiers.

Example: E-commerce uses a web server, application server and database server.

3. Peer-to-peer model

Nodes can act as both clients and servers.

Example: Distributed file sharing and some content-distribution systems.

4. Publish-subscribe model

Publishers send events to a broker or topic and subscribers receive relevant events.

Example: Cloud monitoring systems publish alerts that monitoring applications subscribe to.

5. Distributed data-processing model

Large data is partitioned and processed in parallel.

Example: MapReduce-style analytics processes large web logs across many machines.

6. Cloud service model

Cloud applications commonly use distributed compute, storage and database services.

Example: A web application may use virtual machines, object storage and a managed database.

Importance:

These models provide scalability, resource sharing, fault isolation and location transparency. The suitable model depends on workload, communication pattern, consistency needs and failure assumptions.

QUESTION 3

Illustrate how On-Demand Provisioning works in a cloud environment.

ANSWER (5 MARKS)

On-Demand Provisioning:

On-demand provisioning is the automatic or self-service allocation of cloud resources when a user or application needs them, followed by release when they are no longer required.

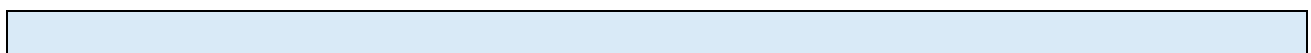
Typical workflow:

User/Application



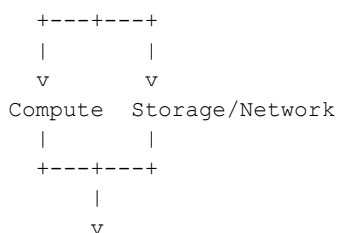
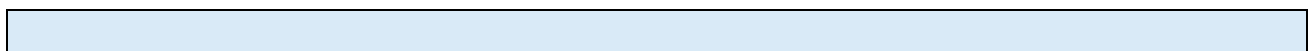
v

Cloud Portal / API



v

Provisioning Controller



Configured Cloud Resource

Steps:

1. A user or application specifies requirements such as CPU, memory, storage, image and network.

2. A request is submitted through a portal, CLI, SDK or API.
3. Identity and authorization policies are checked.
4. The cloud orchestration/provisioning layer selects available resources.
5. Compute, storage and networking resources are allocated.
6. The requested operating system/image and configuration are applied.
7. The resource is connected to required services and made available.
8. Monitoring records utilization and health.
9. When demand decreases, resources can be scaled down or released.

Example:

During an online sale, an application may automatically create additional virtual machines when traffic increases and remove them after the peak.

Benefits:

- Fast deployment
- Elastic scaling
- Self-service
- Better resource utilization
- Pay-per-use efficiency in metered environments

Key idea:

Provisioning creates and configures resources; deprovisioning releases them when they are no longer needed.

QUESTION 4

Define Cloud Computing and list its essential characteristics.

ANSWER (5 MARKS)

Definition:

Cloud Computing is a model for enabling convenient, on-demand network access to a shared pool of configurable computing resources, such as networks, servers, storage, applications and services, that can be rapidly provisioned and released with minimal management effort.

Essential characteristics:

1. On-demand self-service - users can provision resources automatically without direct provider interaction.
2. Broad network access - services are available over networks using standard mechanisms and client platforms.
3. Resource pooling - provider resources are pooled to serve multiple consumers using a multi-tenant model.
4. Rapid elasticity - resources can be rapidly scaled up or down according to demand.
5. Measured service - usage is monitored, controlled and reported for transparency and metering.

Simple view:

Users

v

Cloud Services

+--> Compute
+--> Storage
+--> Network
+--> Applications

Example:

A company can provision virtual machines and storage through a cloud portal instead of purchasing and installing physical servers.

Conclusion:

These characteristics distinguish cloud computing from conventional dedicated infrastructure by emphasizing self-service, pooling, elasticity and measured usage.

QUESTION 5

Discuss the concept of virtualization and explain its significance in cloud computing.

ANSWER (5 MARKS)

Virtualization:

Virtualization is the abstraction of physical computing resources so that multiple logical or virtual environments can share the same physical infrastructure.

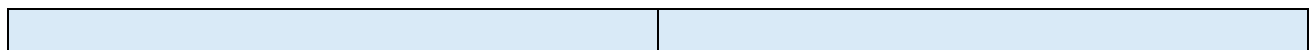
Basic architecture:

Physical Hardware



v

Hypervisor / Virtualization Layer



v v v

VM 1 VM 2 VM 3

OS+A1 OS+A2 OS+A3

The hypervisor manages access to CPU, memory, storage and I/O devices.

Significance in cloud computing:

1. Server consolidation - multiple workloads can run on one physical server.
2. Isolation - virtual machines are logically isolated from one another.
3. Elasticity - virtual machines can be created, resized, migrated or removed more easily.
4. Resource utilization - physical resources can be shared efficiently.
5. Rapid provisioning - standard VM images can be deployed quickly.
6. Fault and workload isolation - failures or heavy workloads can be contained within separate VMs.
7. Migration - workloads can be moved between physical hosts using suitable virtualization technologies.
8. Cost efficiency - fewer physical machines may be required for a given workload.

Types include full virtualization, para-virtualization, hardware-assisted virtualization and OS-level virtualization.

Conclusion:

Virtualization is a key enabling technology for infrastructure clouds because it separates software workloads from physical hardware while providing controlled resource sharing.

QUESTION 6

Differentiate Full Virtualization and Para Virtualization.

ANSWER (5 MARKS)

Full Virtualization and Para Virtualization:

Feature	Full Virtualization	Para Virtualization
Guest OS modification	Usually not required	Guest OS is modified or virtualization-aware
Hardware abstraction	Presents a virtual hardware environment	Uses explicit hypercalls to the hypervisor
Compatibility	High; unmodified operating systems can run	Lower because the guest must support the interface
Performance	Can have virtualization overhead, reduced by hardware assistance	Traditionally lower overhead because guest cooperates
Examples	VMware ESXi, KVM with hardware-assisted virtualization	Xen para-virtualized guests
Use	General-purpose VM hosting	Environments where guest modification is acceptable

Full virtualization:

The hypervisor presents virtual hardware to an unmodified guest OS. Sensitive operations are handled through trapping, binary translation, or hardware-assisted mechanisms.

Para virtualization:

The guest OS is aware that it is virtualized and uses hypercalls to request privileged operations from the hypervisor.

Conclusion:

Full virtualization prioritizes compatibility, while para-virtualization emphasizes cooperation between guest OS and hypervisor for efficient execution.

QUESTION 7

Discuss cloud security challenges and their impact on cloud environments.

ANSWER (5 MARKS)

Cloud security challenges:

1. Data breaches - unauthorized access can expose sensitive information.
2. Misconfiguration - incorrectly configured storage, identity policies or network controls can expose resources.
3. Weak identity and access management - stolen credentials or excessive privileges can enable unauthorized actions.
4. Insecure APIs - cloud management interfaces can become attack surfaces.
5. Multi-tenancy risks - multiple customers share physical infrastructure, requiring strong isolation.
6. Insider threats - privileged users may misuse legitimate access.
7. Data loss - deletion, ransomware, hardware failure or inadequate backup can cause loss.
8. Compliance and data location - organizations must satisfy legal and regulatory requirements.
9. Denial of service - excessive traffic can reduce service availability.
10. Limited visibility - abstraction can make infrastructure-level monitoring more difficult.

Impact:

- Confidentiality loss
- Integrity violations

- Service disruption
- Financial and reputational damage
- Regulatory penalties
- Recovery costs

Basic protection:

Strong IAM, least privilege, encryption, secure configuration, logging, vulnerability management, backups, network segmentation and continuous monitoring are required.

Conclusion:

Cloud security is a shared responsibility. The provider secures the underlying cloud infrastructure, while customers remain responsible for security controls within the scope of the services they use.

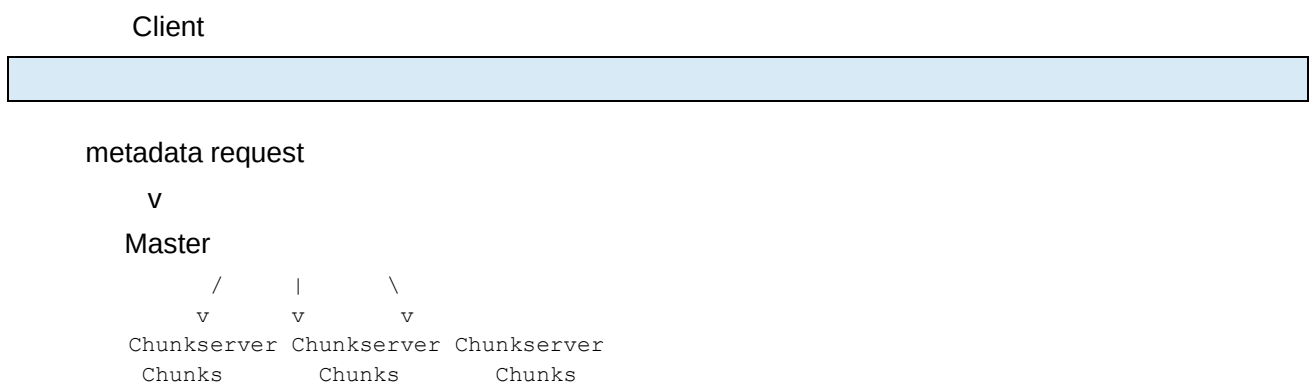
QUESTION 8

Discuss architecture of GFS.

ANSWER (5 MARKS)

GFS (Google File System) is a distributed file system designed for large-scale data-intensive applications. Its classic architecture uses a master and multiple chunkservers.

Architecture:



Main components:

1. Master - maintains file-system metadata such as namespace, file-to-chunk mapping, chunk locations and related control information.
2. Chunkservers - store fixed-size chunks on local disks and serve read/write operations.
3. Clients - contact the master for metadata but normally transfer file data directly with chunkservers.
4. Chunks - large files are divided into chunks, each identified by a unique chunk handle.

Important features:

- Large files are divided into chunks.
- Chunks are replicated for fault tolerance.
- Master manages metadata.
- Data transfer is mainly between clients and chunkservers.
- Heartbeats help the master monitor chunkservers.

The design is optimized for large sequential operations and failure recovery rather than small low-latency transactional workloads.

QUESTION 9

Discuss request/reply protocol and RPC with suitable examples.

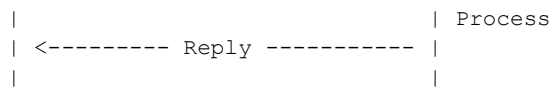
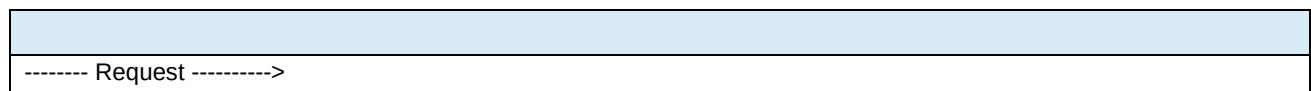
ANSWER (5 MARKS)

Request/Reply Protocol:

The request/reply protocol is a communication pattern in which a client sends a request to a server and the server returns a reply.

Client

Server



Steps:

1. Client identifies the required service.
2. Client constructs a request containing operation and parameters.
3. Request is transmitted to the server.
4. Server receives and processes the request.
5. Server sends a reply containing result or error information.
6. Client receives and uses the response.

RPC (Remote Procedure Call):

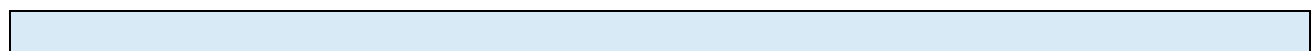
RPC makes a remote operation appear similar to a local procedure call. The client invokes a procedure, while the RPC runtime handles message construction and communication.

Typical RPC path:

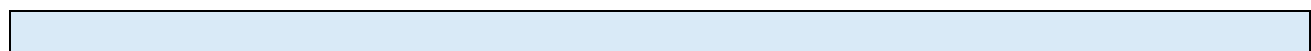
Client



Client Stub



RPC Runtime



Network



RPC Runtime



Server Stub



Server Procedure

Example:

A banking client calls `getBalance(accountNo)`. The client stub marshals the account number into a request. The server stub unmarshals it, invokes the server procedure, and returns the balance.

Important issues:

- Marshalling and unmarshalling
- Request identifiers
- Timeouts

- Retransmission
- Duplicate requests
- Authentication and authorization
- At-most-once or at-least-once semantics depending on implementation

RPC simplifies distributed programming, but remote calls have network latency and failure modes that local procedure calls do not.

QUESTION 10

Explain distributed deadlock detection.

ANSWER (5 MARKS)

Distributed deadlock:

A distributed deadlock occurs when processes located at different nodes wait indefinitely for resources or messages held by one another.

Example:

P1 at Node A holds R1 and waits for R2.

P2 at Node B holds R2 and waits for R1.

```
P1 -> waits for R2
P2 -> waits for R1
=> circular wait => deadlock
```

Detection approaches:

1. Centralized detection - a coordinator collects wait-for information and detects cycles. It is simple but may become a bottleneck or single point of failure.
2. Hierarchical detection - coordinators at different levels combine information.
3. Distributed detection - sites cooperate to detect global wait cycles without a single coordinator.

Wait-for graph:

- Vertex = process
- Directed edge $P_i \rightarrow P_j$ means P_i is waiting for a resource held by P_j .
- A cycle indicates deadlock under the relevant wait-for model.

Distributed detection generally involves:

1. Each site records local wait relationships.
2. Sites exchange dependency information.
3. A probe or dependency message is propagated.
4. The detector determines whether dependencies form a global cycle.
5. If deadlock is confirmed, a recovery action is chosen.

Recovery:

- Abort one or more processes.
- Roll back a selected process.
- Preempt resources when possible.

Challenges:

Message delays, duplicate information, stale dependencies and false detection must be handled carefully.

QUESTION 11

Discuss the evolution of cloud computing and its service models.

ANSWER (5 MARKS)

Evolution of cloud computing:

1. Mainframe and time-sharing - many users shared centralized computing resources.
2. Client-server computing - processing and data services were distributed between clients and servers.
3. Distributed and cluster computing - multiple computers cooperated for performance and availability.
4. Grid computing - geographically distributed resources were coordinated for large computational tasks.
5. Utility computing - computing resources were offered as metered services.
6. Virtualization - workloads were abstracted from physical servers and resources became easier to pool.
7. Cloud computing - on-demand, elastic, measured services became available over networks.

Service models:

IaaS - Infrastructure as a Service provides virtualized compute, storage and networking. Customer manages the OS and applications.

PaaS - Platform as a Service provides a managed application platform/runtime. Customer focuses mainly on application code and data.

SaaS - Software as a Service delivers complete applications to end users; provider manages the underlying platform and infrastructure.

Simple view:

```
SaaS -> complete application
PaaS -> application platform + runtime
IaaS -> virtual infrastructure
```

Cloud computing combines distributed systems, virtualization, automation, networking, storage and service management to provide scalable computing as a service.

QUESTION 12

Analyze the features and use cases of Public, Private and Hybrid cloud deployment models.

ANSWER (5 MARKS)

Cloud deployment models:

Feature	Public Cloud	Private Cloud	Hybrid Cloud
Ownership/operation	Provider	Single organization or dedicated environment	Combination of private and public environments
Resource sharing	Multi-tenant provider infrastructure	Dedicated to one organization	Depends on each environment
Scalability	High	Depends on organization	Can combine private control with public elasticity
Control	Lower over physical infrastructure	Greater control	Mixed
Typical use	General workloads, variable demand	Sensitive or specialized workloads	Workloads needing both environments

Public cloud:

Resources are offered by a cloud provider to multiple customers over a network.

Private cloud:

Cloud infrastructure is dedicated to a single organization and may be operated on-premises or by a third party.

Hybrid cloud:

Two or more distinct cloud environments remain separately managed but are connected to enable data/application portability or coordinated operation.

Use cases:

- Public: web applications and variable workloads.
- Private: internal enterprise systems with specific control requirements.
- Hybrid: sensitive data kept in a controlled environment while public cloud handles elastic processing.

Selection depends on security, compliance, control, cost, performance, portability and workload variability.

QUESTION 13

Explain virtualization tools and mechanisms.

ANSWER (5 MARKS)

Virtualization tools and mechanisms:

Common tools/technologies:

1. Hypervisors - VMware ESXi, Microsoft Hyper-V, KVM and Xen provide virtualization of hardware resources.
2. Container platforms - Docker and container runtimes provide OS-level isolation rather than full hardware virtualization.
3. QEMU - machine emulator/virtualizer commonly used with KVM.
4. VirtualBox - desktop virtualization software used for development and testing.
5. libvirt - management interface for virtualization technologies such as KVM.
6. Virtual machine images - templates containing operating system and software configurations enable repeatable deployment.

Mechanisms:

- CPU virtualization: privileged operations are controlled by the hypervisor and/or hardware virtualization extensions.
- Memory virtualization: guest virtual addresses are mapped to physical memory using mechanisms such as shadow page tables or hardware-assisted nested paging.
- I/O virtualization: devices are emulated, paravirtualized or passed through to guests.
- Isolation: each VM receives controlled access to resources.
- Scheduling: hypervisor schedules virtual CPUs onto physical CPUs.
- Resource allocation: memory, storage and network capacity are assigned to guests.
- Live migration: a running VM can be moved between compatible hosts with limited service interruption.

Importance:

These mechanisms enable consolidation, isolation, rapid provisioning and flexible resource management.

QUESTION 14

Discuss implementation levels of virtualization in detail.

ANSWER (5 MARKS)

Implementation levels of virtualization:

Virtualization can be implemented at different layers of a computing system.

1. Instruction Set Architecture (ISA) level

Software emulates or translates one processor instruction set to another.

Example: an emulator running software built for a different CPU architecture.

2. Hardware abstraction / hardware level

A hypervisor virtualizes physical hardware so multiple VMs can run.

Example: KVM, VMware ESXi, Hyper-V.

3. Operating System level

The kernel provides isolated user spaces that share one host kernel.

Example: containers.

4. Library/API level

A compatibility layer implements or translates application/library interfaces.

Example: compatibility environments that provide expected APIs.

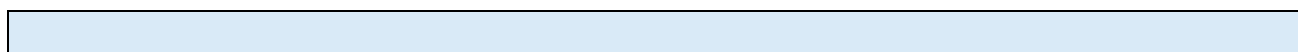
5. Application level

A runtime or virtual machine provides an execution environment for applications.

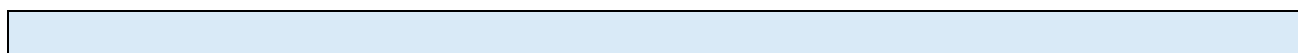
Examples: Java Virtual Machine and managed language runtimes.

Layer view:

Applications



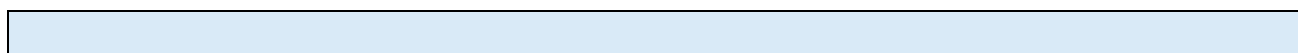
Runtime / Application Virtualization



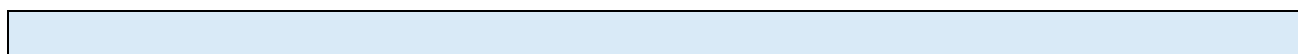
Libraries / API



Operating System



Hardware / Hypervisor



Physical Hardware

Trade-off:

Lower-level virtualization generally provides stronger machine-level isolation and broader OS flexibility, while higher-level approaches can be lighter and faster to start but share more of the underlying environment.

QUESTION 15

Explain data security in cloud environments.

ANSWER (5 MARKS)

Data security in cloud environments:

Data security protects cloud data against unauthorized disclosure, alteration and destruction.

1. Data classification

Classify data as public, internal, confidential or highly sensitive so appropriate controls can be applied.

2. Encryption at rest

Stored data is encrypted on disks, databases or object-storage systems.

3. Encryption in transit

TLS or equivalent secure protocols protect data moving between clients, services and systems.

4. Key management

Use secure key generation, storage, rotation, access control and auditing. Managed key services can reduce operational burden.

5. Identity and access management

Apply authentication, authorization and least privilege. Use multi-factor authentication for sensitive access.

6. Backup and recovery

Maintain protected backups and test restoration.

7. Integrity controls

Hashes, message authentication and digital signatures can help detect unauthorized changes.

8. Data loss prevention

Policies and monitoring can detect and restrict inappropriate movement of sensitive data.

9. Logging and monitoring

Record access, administrative actions and security events.

10. Secure deletion

When data is retired, deletion mechanisms should account for replicas, snapshots and backups according to retention policy.

Conclusion:

Cloud data security requires controls throughout the data life cycle: creation, storage, processing, transmission, sharing, backup and deletion.

QUESTION 16

Discuss Amazon EC2 and its key components.

ANSWER (5 MARKS)

Amazon EC2:

Amazon Elastic Compute Cloud (EC2) is a cloud service that provides resizable compute capacity through virtual servers called instances.

Key components:

1. AMI (Amazon Machine Image) - template used to launch an instance.
2. Instance type - defines a combination of CPU, memory, storage and networking capabilities.
3. EBS (Elastic Block Store) - persistent block storage volumes for instances.
4. Instance store - temporary local storage available for selected instance types.
5. Security groups - virtual firewalls controlling inbound and outbound traffic for instances.
6. Key pairs - cryptographic credentials commonly used for secure instance access.
7. VPC - networking environment in which instances are placed.
8. Elastic IP - static public IPv4 address that can be associated with a resource when required.
9. Auto Scaling - adjusts instance capacity according to demand and policies.
10. Elastic Load Balancing - distributes traffic across targets.

Launch flow:

User -> EC2 API/Console -> Select AMI + Instance Type -> Network/Security -> Launch Instance -> Application

Uses:

- Web and application hosting
- Development and testing

- Batch processing
- Data processing
- Scalable backend services

Benefits include rapid provisioning, elastic capacity and integration with other cloud services.

PART B

15-MARK QUESTIONS

QUESTION 1

Analyze Distributed Mutual Exclusion algorithms and compare their performance.

ANSWER (15 MARKS)

Introduction:

Distributed mutual exclusion ensures that at most one distributed process enters a critical section at a time. Unlike a shared-memory system, processes communicate by messages and there is no single global clock.

Requirements:

- Mutual exclusion
- Deadlock freedom
- Starvation freedom or bounded waiting where required
- Progress
- Fairness
- Reasonable message and synchronization cost

1. Centralized coordinator approach

A coordinator grants permission to one process at a time.

Messages per critical-section entry are typically about 3: request, grant and release.

Advantage: simple.

Limitation: coordinator bottleneck and single point of failure.

2. Lamport's mutual exclusion algorithm

Each request receives a timestamp using Lamport logical clocks. A process multicasts its request to all other processes and enters the critical section when its request is at the head of its local request queue and acknowledgements have been received.

Basic steps:

- Increment logical clock.
- Multicast REQUEST(timestamp, process).
- Every receiver records the request and sends REPLY.
- Process enters when its request is earliest and all required replies have arrived.
- On exit, multicast RELEASE.

For N processes, approximately $3(N-1)$ messages are required per entry in the basic formulation.

3. Ricart-Agrawala algorithm

A process multicasts a REQUEST to every other process. A receiver replies immediately if it is not interested in the critical section or if the requester has priority according to timestamp. Otherwise it defers the reply.

Message cost is approximately $2(N-1)$ messages per entry: N-1 requests and N-1 replies.

4. Token Ring algorithm

Processes are arranged logically in a ring. A unique token circulates. Only the token holder may enter the critical section.

Message cost depends on token position; a process may wait for the token to arrive. There is no request broadcast for every entry.

Comparison:

Algorithm	Main idea	Approx. messages/entry	Main issue
-----------	-----------	------------------------	------------

Centralized	Coordinator grants access	3	Coordinator bottleneck/failure
Lamport	Timestamped request queue	$3(N-1)$	High message traffic
Ricart-Agrawala	Deferred replies	$2(N-1)$	Failure of a participant can block progress
Token Ring	Unique token	Variable	Token loss and ring maintenance

Performance considerations:

- For small systems, centralized coordination is simple.
- Ricart-Agrawala reduces message count compared with Lamport's basic algorithm.
- Token Ring avoids broadcast traffic but may incur token waiting and recovery overhead.
- Fault tolerance, fairness and network topology strongly affect practical performance.

Conclusion:

No algorithm is universally optimal. Selection depends on process count, failure model, network behavior and the required balance between message overhead and coordination simplicity.

QUESTION 2

Evaluate Distributed System Models in real-world applications such as banking, e-commerce, and cloud services.

ANSWER (15 MARKS)

Introduction:

Distributed system models provide architectural patterns for coordinating independent computers. Their usefulness can be evaluated by mapping communication and consistency requirements to real applications.

1. Banking - client-server / multi-tier

Mobile or web clients communicate with banking services through application servers and databases.

Customer App

v

API / Application Tier

v

Transaction Services

v

Database / Ledger

Important properties:

- Strong authentication
- Authorization
- Transaction consistency
- Audit logging
- Fault tolerance

A distributed transaction may update accounts while preserving consistency constraints.

2. E-commerce - multi-tier and service-oriented model

Customer -> Web/API -> Application Services -> Databases/Cache/Storage

Services may include catalogue, cart, order, payment and inventory. Load balancing and horizontal scaling handle variable traffic. Caching improves response time, while asynchronous messaging can decouple services.

3. Cloud services - distributed resource and service models

Cloud providers distribute compute, storage and networking across data centers. A service can scale horizontally by adding instances.

Typical flow:

User -> Internet -> Load Balancer -> Service Instances -> Distributed Storage/Database

4. Peer-to-peer model

Some applications allow nodes to provide services to one another. This can distribute load but introduces discovery and consistency challenges.

5. Publish-subscribe model

Cloud monitoring is a natural example:

Resource events -> Topic/Broker -> Subscribers such as alerting, analytics and automation.

Evaluation criteria:

- Scalability
- Availability
- Consistency
- Latency
- Fault tolerance
- Security
- Operational complexity

Conclusion:

Banking emphasizes correctness, security and transactional consistency; e-commerce emphasizes scalability and responsiveness; cloud services emphasize elasticity, multi-tenancy, automation and fault tolerance. The architecture must match the workload rather than using one model for all applications.

QUESTION 3

Analyze NIST Cloud Computing Reference Architecture, cloud cube model and explain roles of all entities.

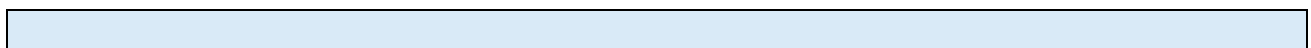
ANSWER (15 MARKS)

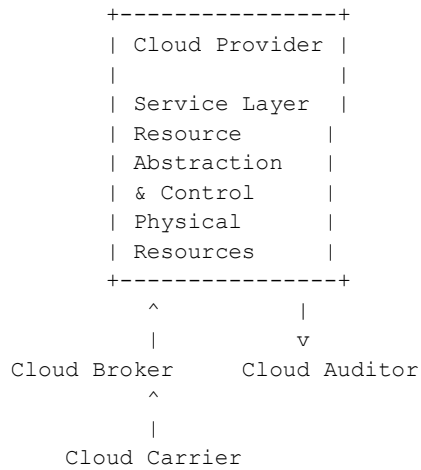
Introduction:

The NIST Cloud Computing Reference Architecture (NIST SP 500-292) provides a conceptual view of cloud actors and their relationships. It is not a deployment blueprint but a reference model for understanding roles and interactions.

Reference architecture:

Cloud Consumer





Major entities:

1. Cloud Consumer

Person or organization that uses cloud services. It selects services, establishes service requirements and consumes IaaS, PaaS or SaaS.

2. Cloud Provider

Organization that makes cloud services available. It manages service orchestration, resource abstraction/control, physical resources and service management.

3. Cloud Broker

Intermediary that manages use, performance and delivery of cloud services. Functions can include service intermediation, aggregation and arbitrage.

4. Cloud Auditor

Independent party that assesses cloud services, security, privacy, performance and compliance.

5. Cloud Carrier

Provides connectivity and transport between cloud consumer and cloud provider.

Cloud service models:

- SaaS: provider delivers applications.
- PaaS: provider delivers application platform capabilities.
- IaaS: provider delivers compute, storage and network resources.

Cloud Cube Model:

The Cloud Cube Model, associated with Jericho Forum, is a framework for analyzing cloud environments using dimensions such as:

- Internal vs external: whether assets are within or outside the organization.
- Proprietary vs open: whether technology is controlled by a single vendor or based on open standards.
- Perimeterized vs de-perimeterized: whether security depends strongly on a defined organizational perimeter.
- Insourced vs outsourced: whether operation is performed by the organization or an external provider.

The cube helps organizations reason about where data/services are located and what security and governance implications arise. It is complementary to, not a replacement for, the NIST reference architecture.

Conclusion:

NIST RA identifies cloud roles and provider components, while the Cloud Cube Model helps analyze cloud characteristics and security context.

QUESTION 4

Evaluate Public, Private, and Hybrid cloud models for a healthcare organization.

ANSWER (15 MARKS)

Introduction:

Healthcare workloads contain sensitive clinical, financial and operational information. Deployment-model selection must consider confidentiality, integrity, availability, compliance, interoperability and cost.

Comparison:

Factor	Public Cloud	Private Cloud	Hybrid Cloud
Control	Provider-managed infrastructure	Greater organizational control	Mixed
Scalability	High	Capacity constrained by provisioned infrastructure	High when public capacity is integrated
Sensitive workloads	Possible with appropriate controls	Often suitable where dedicated control is required	Can separate workloads
Cost model	Usage-based common	Higher ownership/operation burden	Mixed
Flexibility	High	High within owned environment	High across environments
Management	Provider handles much infrastructure	Organization/provider manages dedicated environment	Requires integration and governance

Evaluation for healthcare:

1. Public cloud

Suitable for patient portals, analytics, backup, telemedicine applications and variable workloads when appropriate contractual, security and compliance controls are implemented.

2. Private cloud

May be used for workloads requiring dedicated infrastructure, specialized controls or organizational policies. It can provide greater direct control but may cost more to build and operate.

3. Hybrid cloud

Can keep selected sensitive systems in a controlled environment while using public cloud for elastic analytics, web services or disaster recovery. Secure connectivity, identity federation and data integration are critical.

Required controls:

- Strong IAM and least privilege
- Encryption at rest and in transit
- Audit logs
- Backup and disaster recovery
- Network segmentation
- Secure APIs
- Data classification
- Compliance monitoring
- Incident response

Conclusion:

The appropriate model depends on the healthcare organization's regulatory obligations, data sensitivity, existing infrastructure, workload variability and operational capabilities. A hybrid architecture can combine environments, but it introduces integration and governance complexity.

QUESTION 5

Describe the implementation levels of virtualization and explain CPU, Memory and I/O device virtualization with suitable illustrations.

ANSWER (15 MARKS)

Introduction:

Virtualization can be implemented at multiple levels. In cloud data centers, CPU, memory and I/O virtualization allow several virtual machines to share physical hardware while remaining isolated.

Implementation levels:

1. ISA level

Instruction emulation/translation enables software built for one instruction set to execute on another.

2. Hardware level

A hypervisor provides virtual machines and controls physical CPU, memory and devices.

3. OS level

Containers isolate processes while sharing the host kernel.

4. Library/API level

Software compatibility is provided at an API/library boundary.

5. Application level

A runtime virtual machine provides an execution environment, such as the JVM.

Overall:

Applications -> Guest OS -> Virtual Hardware -> Hypervisor -> Physical Hardware

CPU virtualization:

- A VM sees one or more virtual CPUs.
- The hypervisor schedules virtual CPUs onto physical CPU cores.
- Privileged operations are controlled through hardware support or hypervisor mechanisms.
- CPU time can be allocated using scheduling policies.

Memory virtualization:

Guest Virtual Address

v

Guest Physical Address

v

Host Physical Address

The hypervisor maintains mappings and enforces isolation. Hardware-assisted mechanisms such as nested paging can reduce translation overhead.

I/O virtualization:

Guest OS

Virtual Device

Hypervisor / I/O Layer

Physical Device

Techniques include:

- Device emulation
- Paravirtualized drivers
- Direct device assignment/pass-through
- SR-IOV for suitable hardware

Advantages:

- Resource sharing
- Isolation
- Consolidation
- Rapid provisioning
- Flexible migration

Conclusion:

CPU, memory and I/O virtualization together create the virtual hardware platform on which cloud VMs operate.

QUESTION 6

Analyze server virtualization and desktop virtualization, highlighting their architecture, benefits and applications.

ANSWER (15 MARKS)

Definition:

Server virtualization abstracts physical server resources so multiple virtual servers can run on one physical host. Desktop virtualization separates a user's desktop environment from the endpoint device, often by hosting desktops centrally.

Server virtualization architecture:

Users/Applications

v

Virtual Machines

+-----+-----+

VM 1	VM 2
OS+App	OS+App

+-----+-----+

v

Hypervisor

v

Physical Server Hardware

Benefits:

- Server consolidation
- Better utilization
- Isolation
- Rapid provisioning
- Easier backup and migration
- Reduced hardware footprint

Applications:

Web hosting, development/testing, enterprise servers, disaster recovery and cloud infrastructure.

Desktop virtualization architecture:

User Device



v

Connection Broker



v

Virtual Desktop Infrastructure



```
+++> Desktop VM 1
+++> Desktop VM 2
+++> Desktop VM 3
|
v
```

Shared Storage / Identity / Management

Benefits:

- Centralized desktop management
- Easier patching
- Remote access
- Controlled data location
- Faster replacement of endpoint devices

Applications:

Call centers, labs, education, remote workforce, secure enterprise desktops.

Comparison:

Aspect	Server Virtualization	Desktop Virtualization
Primary target	Server workloads	User desktop environments
Main objective	Consolidate and manage server workloads	Centralize and deliver desktops
Users	Applications/services	Human end users
Key components	Hypervisor, VM, virtual networking/storage	Connection broker, desktop VMs, identity and storage
Typical benefit	Utilization and infrastructure efficiency	Centralized desktop administration

Conclusion:

Both use virtualization and isolation, but their workloads and management goals differ.

QUESTION 7

Discuss cloud security architecture and explain the major cloud security challenges and Data security techniques.

ANSWER (15 MARKS)

Introduction:

Cloud security architecture is a layered set of controls protecting identities, workloads, networks, data and management interfaces. It should follow defense-in-depth and shared-responsibility principles.

Layered architecture:

Users / Devices

v

Identity & Access Management

v

API / Application Security

v

Network Security

v

Compute / Container Security

v

Data Security

v

Logging / Monitoring / Incident Response

Major security challenges:

1. Misconfiguration
2. Weak credentials and excessive privileges
3. Insecure APIs
4. Data breach and data leakage
5. Multi-tenancy/isolation concerns
6. Vulnerable workloads and images
7. Insider threats
8. Availability attacks
9. Compliance and data residency
10. Limited visibility across distributed resources

Data security techniques:

- Encryption at rest and in transit
- Key management and rotation
- IAM and least privilege
- Tokenization or masking where appropriate
- Data classification
- Backups and immutable/protected copies
- Integrity checking
- DLP controls
- Secure deletion
- Continuous monitoring

Security workflow:

Identify assets -> Classify data -> Apply least privilege -> Encrypt -> Monitor -> Detect -> Respond -> Recover

Shared responsibility:

The provider secures the cloud infrastructure according to the service model; the customer secures configurations, identities, applications and data within its responsibility.

Conclusion:

Effective cloud security combines preventive, detective and corrective controls rather than relying on a single security mechanism.

QUESTION 8

Analyze cloud security tools and mechanisms used to identify vulnerabilities and secure cloud resources.

ANSWER (15 MARKS)

Introduction:

Cloud security tools and mechanisms identify vulnerabilities, detect attacks, enforce policy and protect resources throughout their life cycle.

Major tools/mechanisms:

1. Identity and Access Management (IAM)

- Authentication
- Authorization
- Role-based or attribute-based access
- Least privilege
- Multi-factor authentication

2. Vulnerability assessment

Scans hosts, containers, dependencies and configurations for known weaknesses.

3. Security posture/configuration management

Checks resources against secure configuration policies and identifies publicly exposed or overly permissive resources.

4. Web Application Firewall (WAF)

Filters malicious HTTP/S traffic to web applications.

5. Network controls

Firewalls, security groups, network ACLs, segmentation and private connectivity restrict network paths.

6. Intrusion detection/prevention

Monitors traffic or host activity for suspicious behavior.

7. SIEM and centralized logging

Collects and correlates logs from identities, applications, networks and infrastructure.

8. Encryption and key management

Protects data and controls cryptographic keys.

9. Endpoint/workload protection

Monitors virtual machines, containers and other compute workloads.

10. Container/image scanning

Checks images and dependencies for known vulnerabilities before deployment.

Protection workflow:

Asset discovery -> Vulnerability/configuration scan -> Risk assessment -> Remediation -> Continuous monitoring -> Incident response

Example:

Before deploying a containerized application, an organization scans the image, applies least-privilege runtime permissions, restricts network access, encrypts secrets, records logs and continuously monitors runtime behavior.

Conclusion:

Tools are effective when integrated into a continuous security process. Scanning alone does not secure a cloud environment; findings must be prioritized, remediated and monitored.

QUESTION 9

Describe HDFS architecture and explain MapReduce concept with realtime examples.

ANSWER (15 MARKS)

Introduction:

HDFS is a distributed file system designed for reliable, high-throughput storage of large files across clusters of commodity machines. MapReduce is a distributed programming model for processing large data sets in parallel.

HDFS architecture:

Client

metadata request

v

NameNode

/ | \
v v v

DataNode DataNode DataNode
blocks blocks blocks

HDFS components:

1. NameNode - manages namespace and metadata, including file-to-block mapping and block locations.
2. DataNodes - store blocks and serve read/write requests.
3. Secondary/Checkpoint mechanisms - periodically create checkpoints of namespace metadata; it is not simply a backup NameNode.
4. Blocks - large files are divided into blocks and replicated for fault tolerance.
5. Client - contacts NameNode for metadata and communicates with DataNodes for actual data transfer.

HDFS write:

1. Client asks NameNode to create a file.
2. NameNode validates namespace and permissions.
3. Client obtains DataNode targets.
4. Data is split into blocks.
5. Blocks are pipelined to DataNodes.
6. Replicas are written according to the configured replication policy.
7. Acknowledgements flow back to the client.
8. Metadata is updated.

HDFS read:

1. Client asks NameNode for block locations.
2. NameNode returns suitable DataNodes.
3. Client reads blocks directly from DataNodes.

MapReduce:

Input -> Split -> Map -> Shuffle/Sort -> Reduce -> Output

Map:

Processes input records and emits intermediate key-value pairs.

Shuffle and sort:

Groups intermediate values by key and transfers them to reducers.

Reduce:

Aggregates or processes all values for each key and writes final output.

Realtime-style example:

For an e-commerce company, logs may contain:

product_id, user_id, event

A Map function emits (product_id, 1) for each purchase event. Shuffle groups all counts by product_id. Reduce sums the values to obtain purchase totals.

Another example is word count:

```
Map("cloud cloud data") -> (cloud,1), (cloud,1), (data,1)
Shuffle -> cloud:[1,1], data:[1]
Reduce -> cloud:2, data:1
```

Benefits:

- Data locality
- Parallel processing
- Fault tolerance
- Scalability

Conclusion:

HDFS provides distributed storage while MapReduce provides a processing model that can exploit that distributed data.

QUESTION 10

Discuss key features of Google Compute Engine and use cases of Google Compute Engine.

ANSWER (15 MARKS)

Google Compute Engine (GCE):

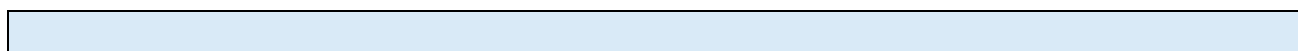
Google Compute Engine is a Google Cloud service that provides configurable virtual machines and related infrastructure resources.

Key features:

1. Customizable VM machine types - CPU and memory configurations can be selected for workload needs.
2. Predefined machine families - optimized for general-purpose, compute, memory or other workloads.
3. Persistent storage - persistent disks can provide durable storage for VM workloads.
4. Networking - VMs operate within Google Cloud networking with configurable IP addressing, firewall rules and load balancing integration.
5. Images - standard and custom images can be used to create VMs.
6. Autoscaling - managed instance groups can adjust capacity for suitable workloads.
7. Load balancing - distributes traffic among instances.
8. Availability options - instances and managed groups can be designed across zones/regions for resilience.
9. GPUs/accelerators - supported workloads can use specialized accelerators where available.
10. Monitoring and logging - integrate with cloud operations services.

Typical architecture:

Users



v

Google Cloud API/Console



v

Compute Engine VMs

```

+--> Persistent Disks
+--> Virtual Network
+--> Load Balancer
+--> Managed Instance Group

```

Use cases:

- Web and application hosting
- Enterprise workloads
- High-performance and batch computing
- Data processing
- Development and testing
- Disaster recovery
- Container or orchestration worker nodes
- Machine learning workloads where supported accelerators are appropriate

Conclusion:

GCE provides flexible VM-based infrastructure with integrated storage, networking, scaling and operations capabilities.

QUESTION 11

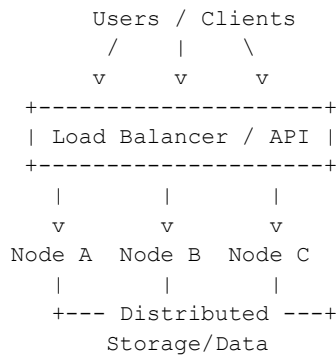
Explain distributed system architecture and characteristics with a neat diagram.

ANSWER (15 MARKS)

Definition:

A distributed system is a collection of independent networked computers that coordinate through communication to provide a coherent service.

Architecture:



Characteristics:

1. Resource sharing - hardware, data and services are shared.
2. Concurrency - multiple processes execute simultaneously.
3. No global clock - each machine has its own local clock.
4. Independent failures - one node may fail while others continue.
5. Message-based communication - components coordinate over a network.
6. Scalability - systems can grow by adding resources.
7. Transparency - the system may hide distribution details such as location or replication.
8. Fault tolerance - redundancy and recovery mechanisms can maintain service.
9. Heterogeneity - different platforms and technologies may interoperate.
10. Security - distributed communication and resources require authentication and authorization.

Architectural forms:

- Client-server
- Multi-tier
- Peer-to-peer
- Service-oriented/microservices
- Distributed data-processing systems

Challenges:

Network latency, partial failure, consistency, synchronization, naming, concurrency, security and load balancing.

Conclusion:

Distributed architecture improves resource sharing, scalability and availability but requires careful coordination because there is no shared memory or perfect global time.

QUESTION 12

Discuss election algorithms (Bully and Ring) with examples.

ANSWER (15 MARKS)

Introduction:

Election algorithms select a coordinator among distributed processes when the current coordinator fails or a new coordinator is needed. The Bully and Ring algorithms are classic approaches.

1. Bully algorithm

Assumption:

Each process has a unique numeric identifier. The process with the highest identifier among participating processes becomes coordinator.

Example:

Processes: P1, P2, P3, P4, P5

P5 is coordinator.

If P5 fails and P2 detects the failure:

```
P2 -> ELECTION -> P3, P4, P5
P3/P4 respond "OK" if alive
```

Higher process continues election

P4 -> COORDINATOR -> all lower processes

Steps:

1. A process detects coordinator failure.
2. It sends ELECTION messages to all processes with higher IDs.
3. If no higher process responds, it declares itself coordinator.
4. If a higher process responds, that process takes over the election.
5. The highest surviving process announces COORDINATOR.

Advantages:

- Simple concept
- Quickly selects highest-ID surviving process under assumptions

Limitations:

- Many messages can be generated, especially with several simultaneous initiators.
- Requires knowledge of higher-ID processes and reliable failure detection.

2. Ring algorithm

Processes are arranged in a logical ring.

```
P1 -> P2 -> P3 -> P4 -> P5
^                               |
+-----+
|
```

If a process detects coordinator failure:

1. It creates an election message containing its identifier.
2. The message travels around the ring.
3. Each process adds or forwards the relevant identifier.
4. After the message completes the ring, the highest active identifier is selected.
5. A coordinator message circulates to announce the winner.

Advantages:

- Structured communication
- No need to contact every process directly

Limitations:

- Ring must be maintained when nodes fail.
- Election can take multiple message hops.

Comparison:

Feature	Bully	Ring
Organization	Identifier-based	Logical ring
Election communication	To higher-ID processes	Around ring
Winner	Highest surviving ID	Highest surviving ID
Main overhead	Potentially many messages	Ring traversal
Failure handling	Requires membership/failure knowledge	Requires ring maintenance

Conclusion:

Bully can react directly to higher-ID processes, while Ring uses ordered circulation of election messages.

QUESTION 13

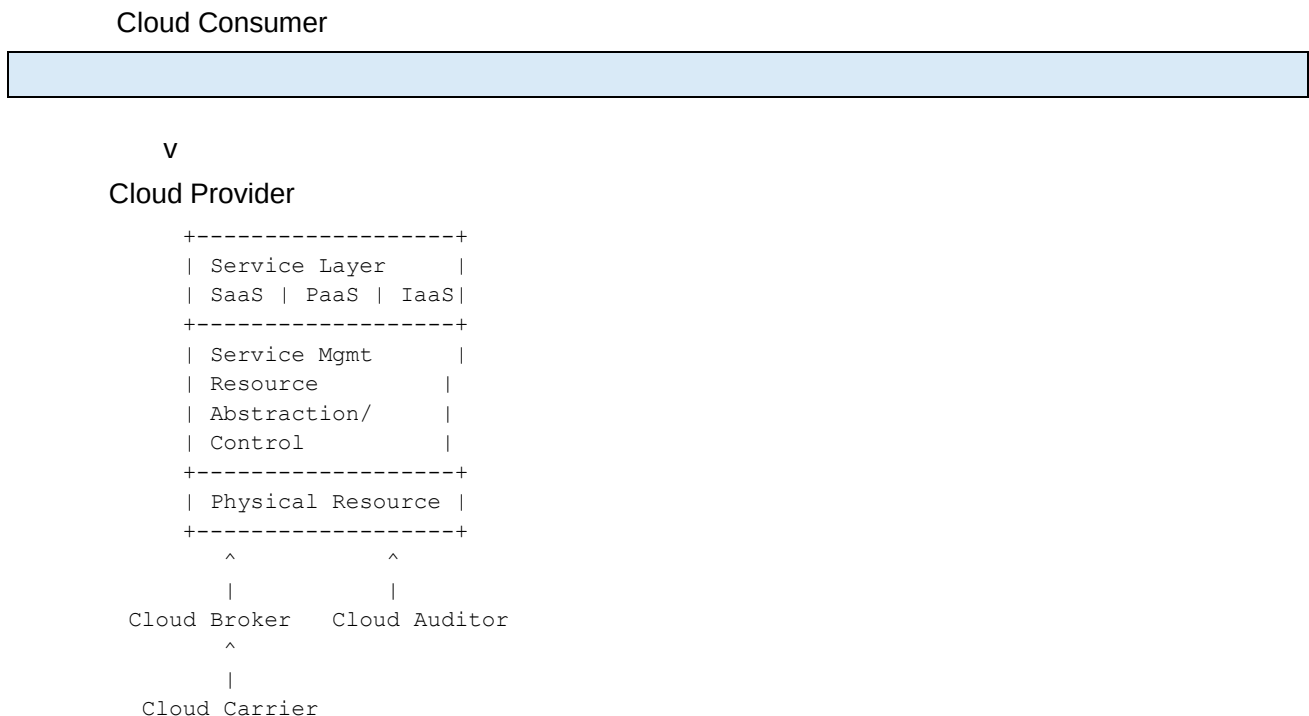
Describe the NIST cloud computing reference architecture in detail.

ANSWER (15 MARKS)

NIST Cloud Computing Reference Architecture:

The NIST reference architecture provides a vendor-neutral conceptual model for understanding cloud roles, service orchestration and management.

High-level view:



1. Cloud Consumer

Uses cloud services. Responsibilities include selecting services, defining requirements and managing consumer-side configurations.

2. Cloud Provider

Makes cloud services available. Its architecture includes:

- Service orchestration
- Service layer
- Resource abstraction and control layer
- Physical resource layer
- Cloud service management

3. Cloud Broker

Intermediates between consumers and providers.

Functions:

- Service intermediation
- Service aggregation
- Service arbitrage

4. Cloud Auditor

Performs independent assessment of security, privacy, performance and compliance.

5. Cloud Carrier

Provides network connectivity and transport.

Service orchestration:

The provider coordinates compute, storage, networking, virtualization and software components to deliver the requested service.

Cloud service models:

IaaS - virtual infrastructure.

PaaS - managed application platform.

SaaS - complete application.

Cloud service management includes:

- Business support
- Provisioning/configuration
- Portability and interoperability
- Security and privacy management

Importance:

The architecture helps organizations identify responsibilities, interfaces and management concerns without prescribing a particular vendor implementation.

Conclusion:

The NIST model is a conceptual reference architecture that organizes cloud actors and provider components and clarifies how cloud services are delivered.

QUESTION 14

Compare IaaS, PaaS and SaaS with real-world examples.

ANSWER (15 MARKS)

Introduction:

IaaS, PaaS and SaaS are the three standard cloud service models. They differ mainly in the level of abstraction and the amount of infrastructure managed by the provider.

Comparison:

Feature	IaaS	PaaS	SaaS
Main offering	Virtual infrastructure	Application platform	Complete application
Customer manages	OS, apps, data and configurations	Application code and data	Mainly application use and data/configuration within product
Provider manages	Physical infrastructure and virtualization plus service components	Infrastructure, OS/runtime/platform	Entire application stack and infrastructure
Flexibility	High	Medium	Lower at infrastructure level
Typical user	Infrastructure/admin teams	Developers	End users/business teams
Example	Amazon EC2	Google App Engine	Salesforce

IaaS:

Provides virtual machines, storage, networking and related infrastructure. The customer installs/configures the operating system and applications.

PaaS:

Provides managed runtime, middleware and development services so developers can focus on application code.

SaaS:

Provides a finished application accessed over a network, commonly through a browser or API.

Responsibility view:

SaaS: Provider manages most of the stack.

PaaS: Customer focuses on application and data.

IaaS: Customer manages more of the software stack.

Real-world examples:

- IaaS: Amazon EC2
- PaaS: Google App Engine
- SaaS: Salesforce

Conclusion:

The choice depends on desired control, operational responsibility, development speed and application requirements.

QUESTION 15

Explain CPU, memory and I/O virtualization with diagrams.

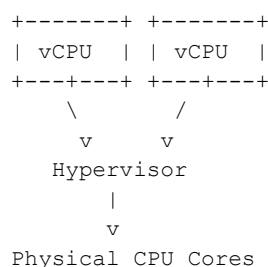
ANSWER (15 MARKS)

Definition:

Virtualization abstracts physical resources so multiple logical execution environments can share one physical system while maintaining controlled isolation.

1. CPU virtualization

Virtual Machines



The hypervisor schedules virtual CPUs on physical cores and controls privileged operations. Hardware-assisted virtualization can improve performance and isolation.

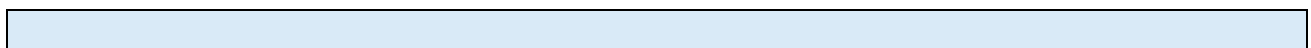
2. Memory virtualization

VM Virtual Address



v

Guest Physical Address



v

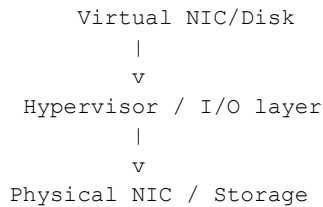
Host Physical Memory

Each VM sees its own address space. The hypervisor and processor memory-translation mechanisms map guest memory to host physical memory while preventing one VM from accessing another.

3. I/O virtualization

Guest OS





Methods:

- Device emulation: software presents a virtual hardware device.
- Paravirtualized I/O: guest uses virtualization-aware drivers.
- Device pass-through: a physical device is assigned more directly to a VM.
- SR-IOV: supported hardware can expose virtual functions for efficient I/O sharing.

Importance:

CPU, memory and I/O virtualization are the core mechanisms that make general-purpose VM-based cloud infrastructure possible.

Conclusion:

Together they provide resource sharing, isolation, flexible allocation and workload mobility.

Scheme of Evaluation:

Definition – 2 marks, Explanation – 10 marks, Diagram – 3 marks.

QUESTION 16

Describe desktop and server virtualization with use cases.

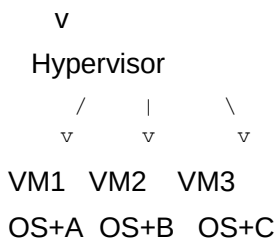
ANSWER (15 MARKS)

Definition:

Server virtualization divides a physical server into multiple isolated virtual machines. Desktop virtualization delivers user desktop environments from centralized or hosted infrastructure.

Server virtualization:

Physical Server



Use cases:

- Web servers
- Application servers
- Database environments where supported
- Development and testing
- Server consolidation
- Disaster recovery
- Cloud infrastructure

Desktop virtualization:

User Device



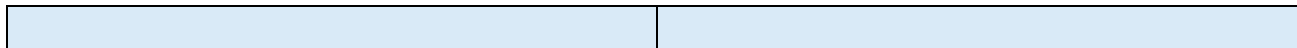
v

Connection Broker



v

Desktop VM Pool



v v v

D1 D2 D3

Use cases:

- Remote workforce
- Education and laboratories
- Call centers
- Controlled enterprise desktops
- Centralized application delivery

Key differences:

Aspect	Server Virtualization	Desktop Virtualization
Workload	Server/service	User desktop
Main goal	Consolidation and scalable services	Centralized desktop delivery
Typical control	Infrastructure/application teams	IT/end-user computing teams
User interaction	Usually indirect	Interactive desktop session

Benefits:

Server: utilization, isolation, provisioning, migration.

Desktop: centralized management, remote access, controlled data location.

Conclusion:

Both rely on virtualization but optimize different workload classes.

Scheme of Evaluation:

Definition – 2 marks, Explanation – 5 marks, Use cases – 8 marks.

QUESTION 17

Discuss cloud security tools and mechanisms for protecting cloud resources.

ANSWER (15 MARKS)

Definition:

Cloud security tools and mechanisms are technologies and controls used to discover risks, prevent unauthorized access, protect workloads/data, detect incidents and support recovery.

Security tools:

1. IAM - authentication, authorization, roles, policies and least privilege.
2. MFA - strengthens identity security.
3. Firewalls/security groups - restrict network traffic.
4. WAF - protects web applications from malicious requests.

5. Vulnerability scanners - identify known software and configuration weaknesses.
6. Cloud security posture management - identifies risky configurations and policy violations.
7. SIEM/log management - collects and correlates security events.
8. IDS/IPS - detects or blocks suspicious traffic.
9. Key management/encryption services - protect data.
10. Container/image scanners - identify vulnerable packages and images.

Protecting cloud resources:

- Apply least privilege.
- Use MFA for privileged accounts.
- Segment networks and restrict inbound access.
- Encrypt sensitive data at rest and in transit.
- Store secrets in managed secret/key services rather than source code.
- Patch operating systems, libraries and applications.
- Scan infrastructure and images before deployment.
- Enable centralized logging and continuous monitoring.
- Maintain tested backups and disaster recovery.
- Apply secure configuration baselines.
- Use automated policy enforcement where practical.
- Prepare incident response procedures.

Security lifecycle:

Discover -> Assess -> Protect -> Monitor -> Detect -> Respond -> Recover

Conclusion:

Cloud protection requires layered controls and continuous assessment; no single security product is sufficient.

Scheme of Evaluation:

Definition – 2 marks, Security tools – 5 marks, Protecting cloud resources techniques – 8 marks.

QUESTION 18

Explain cloud storage architecture and security mechanisms in detail.

ANSWER (15 MARKS)

Definition:

Cloud storage architecture is the organization of storage services, data paths, metadata, replication, access controls and management components that provide scalable and durable storage over a network.

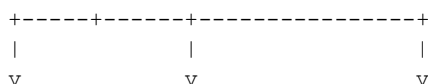
Generic architecture:

Clients / Applications



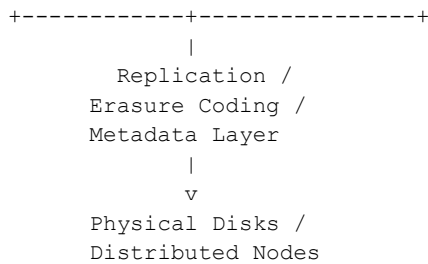
v

Storage API / Gateway



Object Block File

Storage Storage Storage



Storage types:

1. Object storage - stores data as objects with metadata and identifiers; suited to unstructured data, backups and media.
2. Block storage - provides block devices for VM and database workloads.
3. File storage - provides hierarchical files/directories using shared file-system semantics.

Architecture components:

- Storage clients/APIs
- Metadata service
- Storage nodes
- Replication or erasure coding
- Monitoring and management
- Access-control layer

Security mechanisms:

1. Authentication and authorization
2. Encryption at rest
3. Encryption in transit
4. Key management and rotation
5. Access policies and least privilege
6. Versioning and protected deletion
7. Backup and disaster recovery
8. Logging and auditing
9. Data classification and lifecycle policies
10. Integrity checking

Applications:

- Object storage: backup, media, data lakes.
- Block storage: VM disks and transactional workloads.
- File storage: shared application data and enterprise file services.

Conclusion:

A secure cloud storage design combines scalable storage architecture with identity, encryption, integrity, logging, backup and lifecycle controls.

Scheme of Evaluation:

Definition – 2 marks, Storage architecture – 5 marks, Security mechanisms with applications – 8 marks.

QUESTION 19

Describe the architecture of Google File System (GFS) and explain its working with suitable diagrams.

ANSWER (15 MARKS)

Definition:

Google File System (GFS) is a distributed file system designed by Google for large-scale, data-intensive applications. Its classic design emphasizes high throughput, fault tolerance and operation on large files.

Architecture:

Client



metadata requests

v

Master

/

|

\

v

v

v

Chunkserver

Chunkserver

Chunkserver

[chunks]

[chunks]

[chunks]

|

|

|

replicas

replicas

replicas

Components:

1. Master

Maintains namespace and metadata such as file-to-chunk mappings, chunk handles and chunk locations.

2. Chunkservers

Store chunks on local disks and serve read/write operations.

3. Client

Requests metadata from the master but transfers file data directly with chunkservers.

4. Chunks

Large files are divided into fixed-size chunks. Each chunk has a unique handle and is replicated.

Working - read:

1. Client converts a file name and byte range into the required chunk.
2. Client contacts master for the chunk handle and locations.
3. Master returns suitable chunkserver locations.
4. Client contacts a chunkserver directly and reads data.

Working - write:

1. Client asks master for chunk information.
2. Master identifies replicas.
3. Client obtains the primary replica and secondary replicas.
4. Data is sent to replicas.
5. Client sends a write request to the primary.
6. Primary determines the serial order of the mutation.
7. Primary forwards the mutation order to secondary replicas.
8. Replicas acknowledge completion.
9. Primary reports the result to the client.

Fault tolerance:

- Multiple chunk replicas.
- Master monitors chunkservers.
- Heartbeats provide status information.
- Failed replicas can be re-replicated.

Example:

A large log file is split into chunks. A client reads different ranges directly from chunkservers rather than routing all data through the master.

Key design point:

The master manages metadata, while bulk data transfer bypasses the master. This prevents the master from becoming the data path bottleneck.

Conclusion:

GFS achieves scalable, fault-tolerant storage by combining centralized metadata management with distributed chunk storage and replication.

Scheme of Evaluation:

Definition – 2 marks, Architecture – 5 marks, Working with examples – 8 marks.

QUESTION 20

Discuss Docker container architecture, features, advantages, and real-world applications.

ANSWER (15 MARKS)

Definition:

Docker is a container platform that packages an application and its dependencies into portable containers. Containers isolate processes while sharing the host operating system kernel.

Docker architecture:

Developer / User

v

Docker CLI / API

v

Docker Engine

v v v

Images Containers Networks/Volumes

v v

Registry Host Kernel

v

Hardware

Main components:

1. Docker client - command-line or API interface used to request operations.
2. Docker daemon/engine - manages images, containers, networks and volumes.
3. Images - read-only layered templates used to create containers.
4. Containers - running instances of images.
5. Registry - stores and distributes images.
6. Volumes - persistent data storage.
7. Networks - provide container connectivity.

Features:

- Lightweight process isolation
- Layered images
- Portability
- Fast startup
- Versioned image distribution
- Declarative builds using Dockerfiles
- Network and storage abstractions
- Resource limits and isolation through OS mechanisms

Working:

1. Developer writes a Dockerfile.
2. Docker builds an image from the Dockerfile.
3. Image is stored locally or pushed to a registry.
4. A container is created from the image.
5. The container runs the application using the host kernel.
6. Volumes and networks are attached when required.
7. The container can be stopped, recreated or scaled by an orchestration system.

Advantages:

- Consistent development/test/deployment environments
- Faster startup than full VMs in many cases
- Efficient resource use
- Portable application packaging
- Isolation
- Easy CI/CD integration

Limitations:

- Containers share the host kernel, so isolation differs from a full VM.
- Persistent state requires deliberate storage design.
- Image vulnerabilities and insecure configurations must be managed.
- Networking and orchestration can become complex at scale.

Applications:

- Microservices
- CI/CD pipelines
- Development and testing
- Web application deployment
- Batch processing
- Cloud-native applications
- Reproducible software environments

Example:

A web service can be packaged with its runtime and dependencies into an image. The same image can be tested in CI and deployed to a compatible container environment.

Conclusion:

Docker standardizes application packaging and deployment through lightweight OS-level virtualization, making it an important technology for cloud-native software delivery.

Scheme of Evaluation:

Definition – 2 marks, Architecture & Features – 6 marks, Advantages/Applications – 7 marks.

FINAL REVISION CHECKLIST

Important definitions

- ☐ Distributed system
- ☐ Cloud Computing
- ☐ Virtualization
- ☐ On-demand provisioning
- ☐ Full virtualization and para-virtualization
- ☐ RPC and request/reply protocol
- ☐ Distributed deadlock
- ☐ HDFS and GFS
- ☐ IaaS, PaaS and SaaS
- ☐ Server and desktop virtualization
- ☐ Cloud storage
- ☐ Docker containers

Important architectures

- ☐ Generic distributed-system architecture
- ☐ NIST Cloud Computing Reference Architecture
- ☐ Cloud Cube Model dimensions
- ☐ Virtualization stack
- ☐ CPU, memory and I/O virtualization
- ☐ HDFS architecture
- ☐ GFS architecture
- ☐ Cloud storage architecture
- ☐ Docker architecture
- ☐ EC2 component view
- ☐ Google Compute Engine component view

Important diagrams to practise

- ☐ On-demand provisioning workflow
- ☐ Hypervisor with multiple VMs
- ☐ Request/reply and RPC flow
- ☐ Distributed deadlock wait-for graph
- ☐ Distributed mutual exclusion flow
- ☐ Bully election
- ☐ Ring election
- ☐ MapReduce pipeline
- ☐ Server virtualization
- ☐ Desktop virtualization
- ☐ Cloud security layered architecture
- ☐ CPU, memory and I/O virtualization

Important formulas / quantitative facts

- ☐ Lamport mutual exclusion: approximately $3(N-1)$ messages per critical-section entry in the basic algorithm.

- ☐ Ricart-Agrawala: approximately $2(N-1)$ messages per entry.
- ☐ Remember that token-ring message cost depends on token position and ring behavior rather than a fixed broadcast count.
- ☐ Use the exact variable definitions and assumptions when writing message-complexity expressions.

Important algorithms / workflows

- ☐ On-demand provisioning and deprovisioning
- ☐ Distributed deadlock detection using wait-for dependencies
- ☐ Lamport mutual exclusion
- ☐ Ricart-Agrawala mutual exclusion
- ☐ Token Ring mutual exclusion
- ☐ Bully election algorithm
- ☐ Ring election algorithm
- ☐ HDFS read and write paths
- ☐ MapReduce: Map -> Shuffle/Sort -> Reduce
- ☐ GFS read and write paths
- ☐ Cloud security lifecycle: Discover -> Assess -> Protect -> Monitor -> Detect -> Respond -> Recover
- ☐ Docker image build -> registry -> container creation -> execution

Important comparisons

- ☐ Full Virtualization vs Para Virtualization
- ☐ Public vs Private vs Hybrid cloud
- ☐ IaaS vs PaaS vs SaaS
- ☐ Server vs Desktop virtualization
- ☐ Centralized vs Lamport vs Ricart-Agrawala vs Token Ring mutual exclusion
- ☐ Bully vs Ring election
- ☐ Object vs Block vs File cloud storage
- ☐ VM virtualization vs containerization

High-priority concepts to revise before the exam

- ☐ Five essential characteristics of cloud computing
- ☐ Cloud deployment and service models
- ☐ NIST cloud reference architecture and all five cloud actors
- ☐ Virtualization implementation levels
- ☐ CPU, memory and I/O virtualization
- ☐ Distributed mutual exclusion and election algorithms
- ☐ Distributed deadlock detection
- ☐ HDFS and MapReduce
- ☐ GFS architecture and working
- ☐ Cloud security architecture, challenges and data security
- ☐ Cloud storage security
- ☐ EC2 and Google Compute Engine
- ☐ Docker architecture, features, advantages and applications